

Diagram illustrating the PL-App development workflow:

- PL-App**: The user interface for running and monitoring applications, showing code and a graph.
- PL-App Launcher**: The central control interface for managing devices and applications. It includes a list of devices and buttons for **Discover devices**, **Launch**, and **Stop**.
- PL-App Image**: The application image, represented by a green circuit board.
- LAN**: The network connecting the launcher to the application image.
- Discovery of devices**: The process of finding available devices on the LAN.
- Board Provisioning, List of devices, Login to Devices**: The sequence of actions performed by the launcher to prepare the device for application execution.
- Jupyter Notebooks: Labs, Code examples, Dev environment**: The development environment used by the user to create and test applications.

- Part 3: (Challenge) **Implement an International Morse code Help Signaler**

using the **GPIO** ports on the Raspberry Pi. In Part 1, you will build a simple circuit with an external LED that is connected to the **GPIO** ports on the Raspberry Pi. In **Part 2**, you will use the **PL-App** visual programming tool, **Blockly**, to create a program that will blink the LED when uploaded and run on the Raspberry Pi.

Part 3 challenges you to modify the **Blockly** code to have the LED blink the International Morse code signal for help.

- Raspberry Pi with power cable and either a wired or wireless network connection
- Raspberry Pi that is configured and imaged for PL-App access

Raspberry Pi 3 Model B (J8 Header) pinout

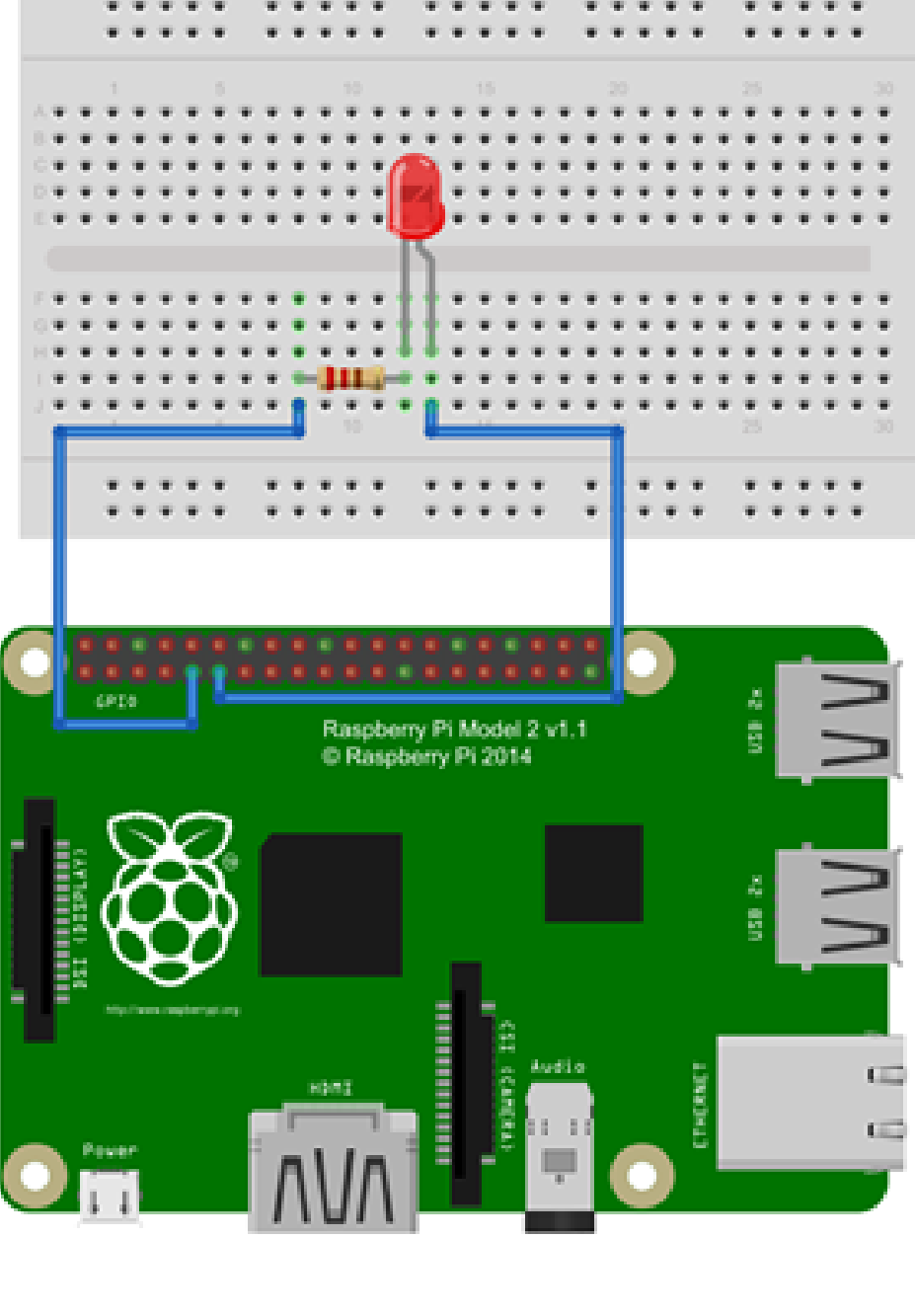
Pin Number	BCM Pin	Physical Pin	Physical Pin	BCM Pin	Pin Name	Pin Name
	3v3 Power	4		5	5v Power	
8	BCM 2 (SDA)	9		4	5v Power	
9	BCM 3 (SCL)	5		6	Ground	
7	BCM 4 (GPCLK0)	7		8	BCM 14 (TXD)	15
	Ground	8		10	BCM 15 (RXD)	16
0	BCM 17	11		12	BCM 18 (PCM_C)	1
2	BCM 27 (PCM_D)	13		14	Ground	
3	BCM 22	15		16	BCM 23	
	3v3 Power	17		18	BCM 24	5
12	BCM 10 (MOSI)	19		20	Ground	
13	BCM 9 (MISO)	21		22	BCM 25	6
14	BCM 11 (SCLK)	23		24	BCM 8 (CE0)	10
	Ground	25		26	BCM 7 (CE1)	11
	BCM 0 (ID_SD)	27		28	BCM 1 (ID_SC)	
21	BCM 5	29		30	Ground	
22	BCM 6	31		32	BCM 12	26
23	BCM 13	33		34	Ground	
24	BCM 19 (MISO)	35		36	BCM 16	27
25	BCM 26	37		38	BCM 20 (MOSI)	28
	Ground	39		40	BCM 21 (SCLK)	29

Note: The Raspberry Pi should not be powered on when building the prototyping circuit.

1. Select 1 – 330 Ohm (Ω) resistor, 1 LED, 1 red jumper wire, 1 black jumper wire from the Raspberry Pi starter kit.
2. Insert the LED onto the breadboard. The cathode leg is connected to c13 and the anode leg is to be connected c14.
3. Insert the resistor onto the breadboard into the b9 and b13 connectors.

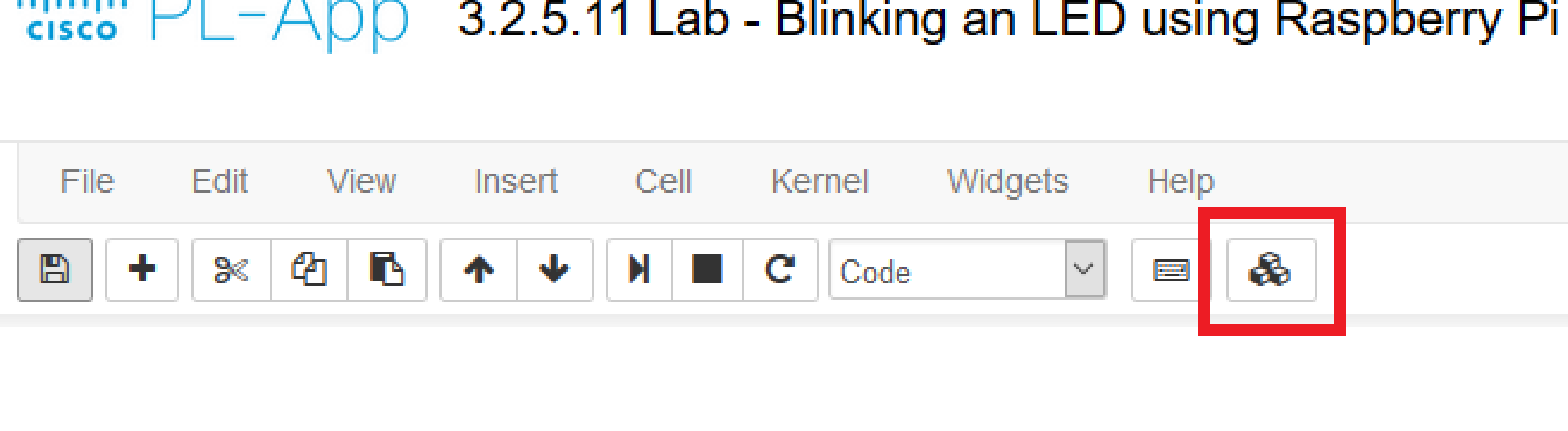
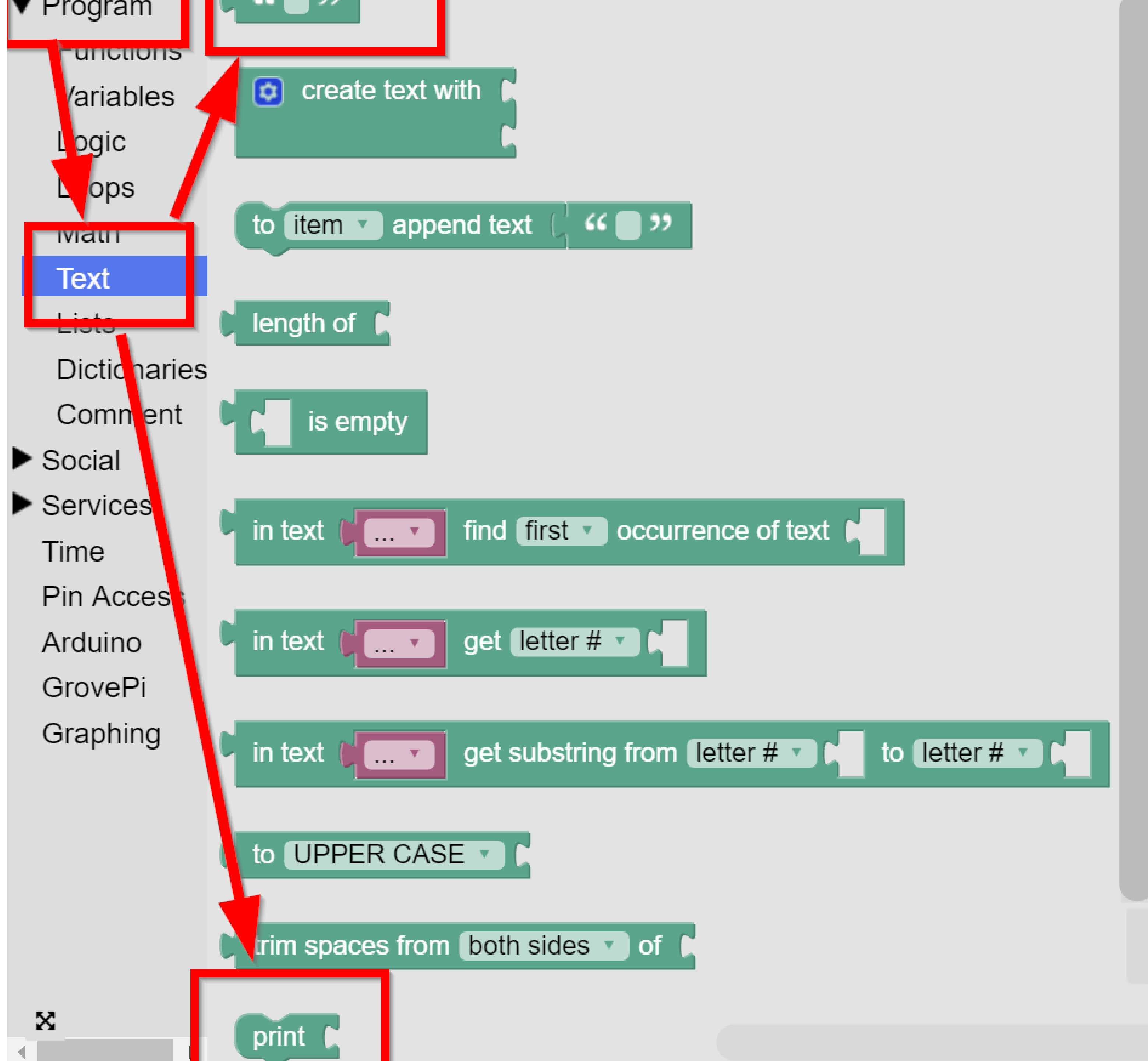
Note: The GPIO pins on the Raspberry Pi are male. If available, use male to Female jumper wires to connect from the breadboard to the Raspberry Pi. Otherwise, use the GPIO ribbon cable, provided in the starter kit, to convert the GPIO pins to female connectors.

1. Connect one end of the black jumper wire to the breadboard a9 connector and the other end to GPIO Physical pin 9 on the Raspberry Pi.
2. Connect one end of the red jumper wire to the breadboard a14 connector and the other end to GPIO Physical pin 11 (BCM17) on the Raspberry Pi.

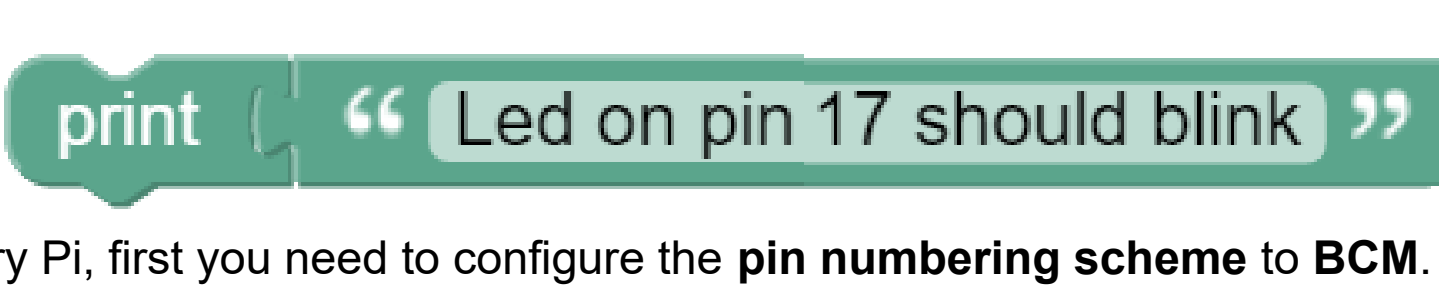


Step 1: Create a Visual (Blockly) Programming cell.

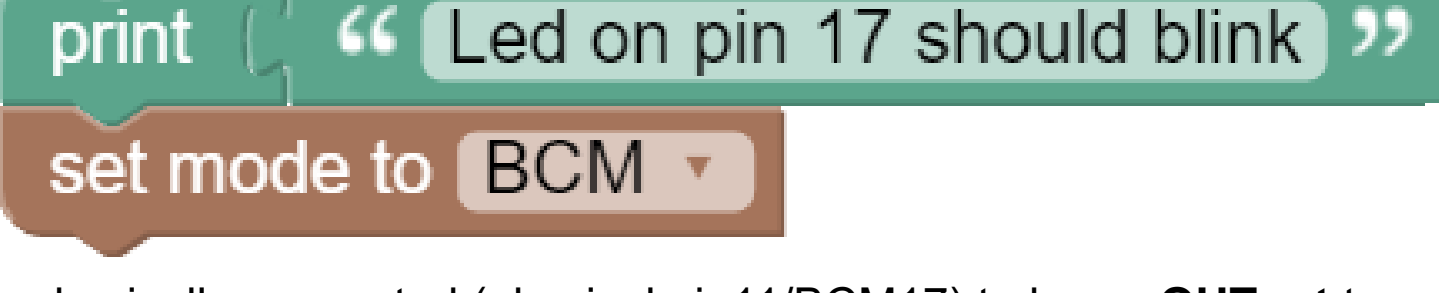
- d. Enter the code cell below and turn it from a default Python code cell to a Visual Programming cell by clicking on the **Blockly icon** in the toolbar

[illegible]

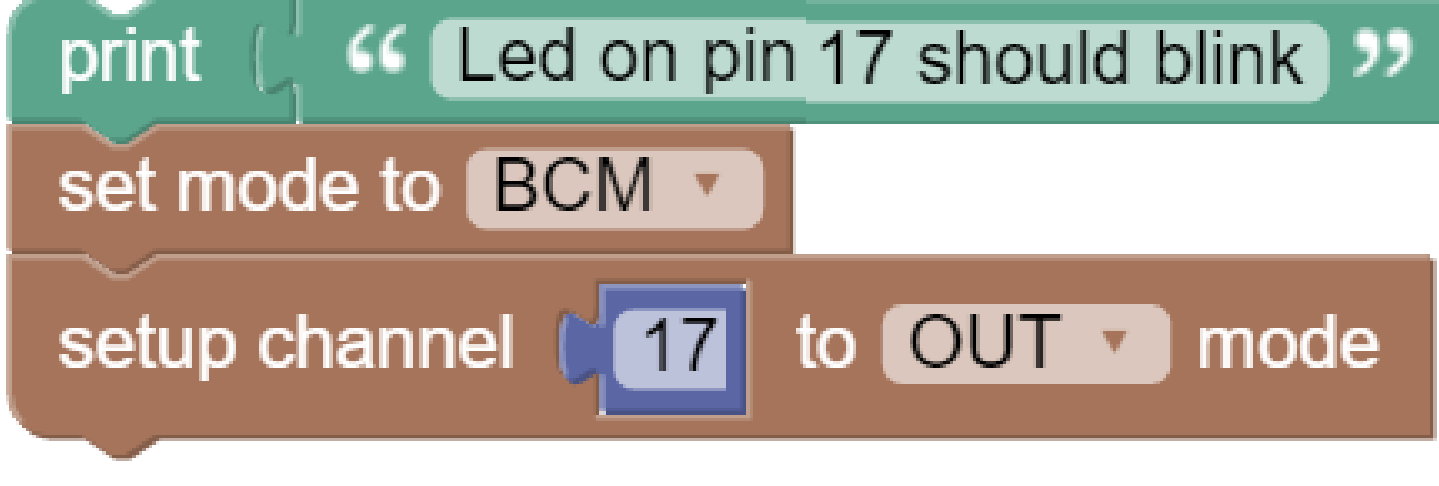
2. Connect the **Text** block to the **Print** block. Add to the **Text** block the following text: Led on pin BCM 17 should blink



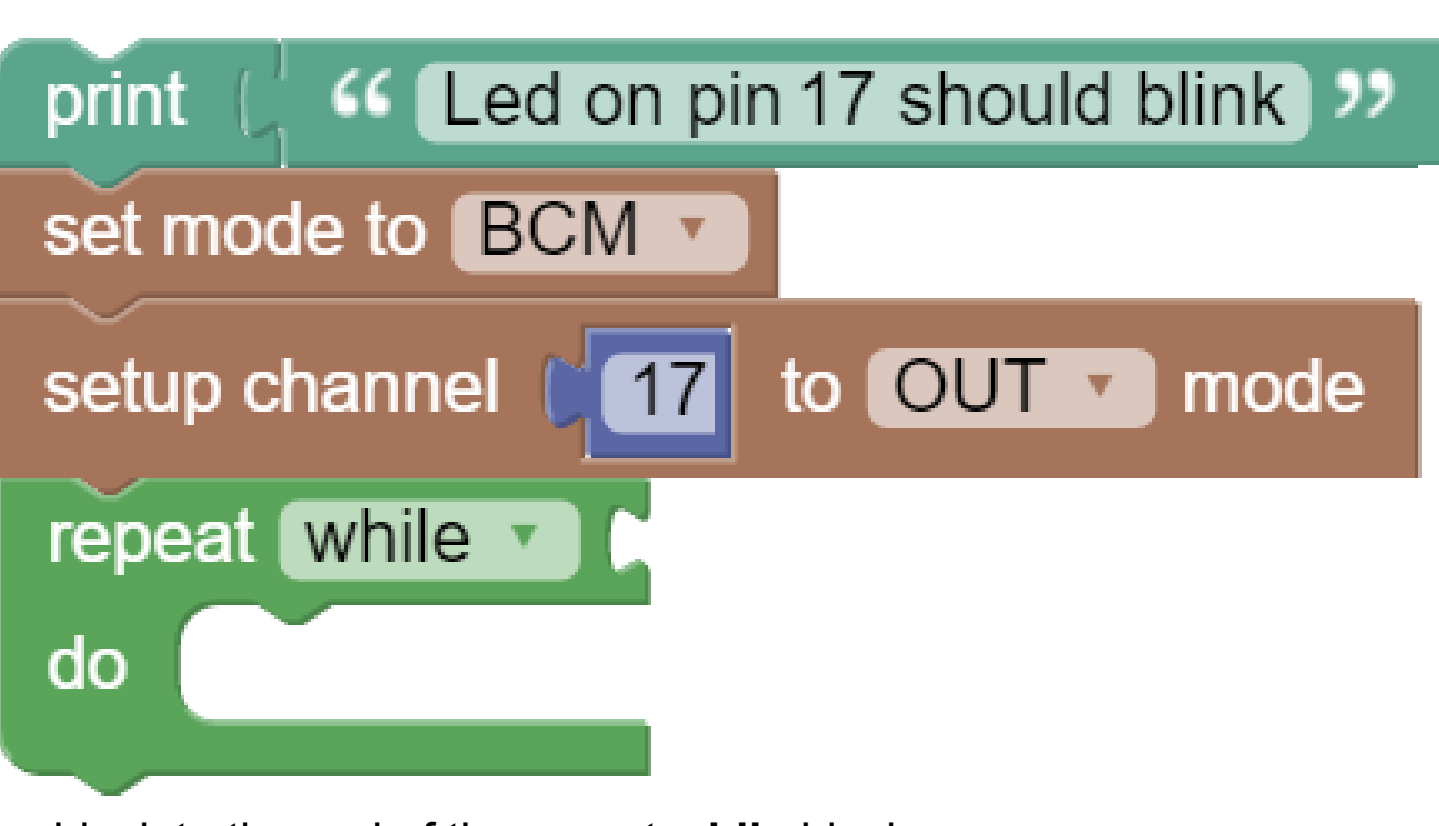
3. To control the GPIO pins of the Raspberry Pi, first you need to configure the **pin numbering scheme** to **BCM**. From the Blockly **Pin Access** menu, select the **set mode to block** and add right after the **print** block. Change its value to **BCM**.



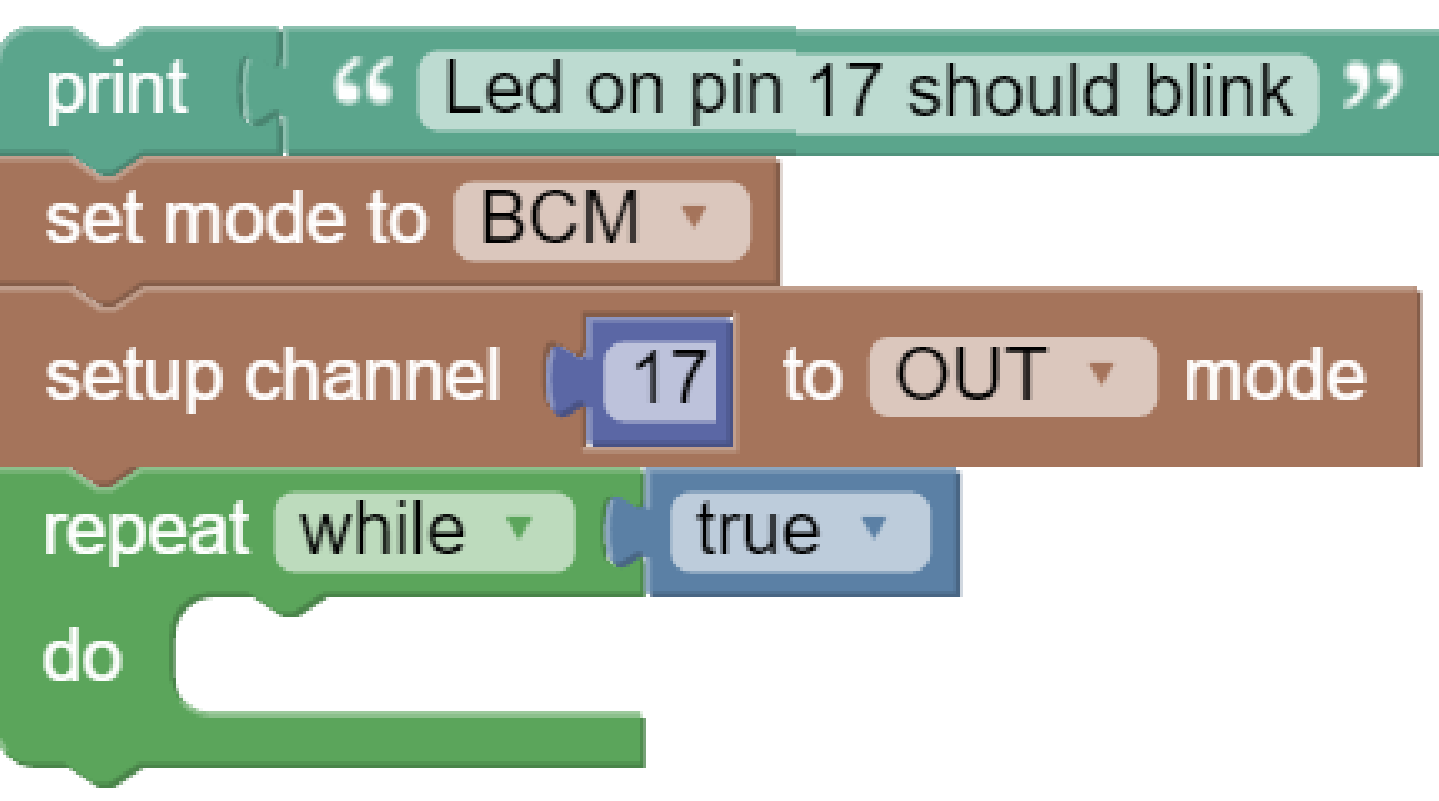
4. Configure the GPIO pin where the LED is physically connected (physical pin1/BCM17) to be an **Output** type. Select the **setup channel** block from the **Pin Access** menu and add it to the canvas. The first number - the channel - represents the GPIO pin (with the specified pin numbering scheme - defined with the block before to be BCM) you want to configure (BCM17), while the second parameter is the direction - **Output** or **Input** - of that pin. Since we want to control the state of the LED from the Raspberry Pi, the mode is in this case **Output**.



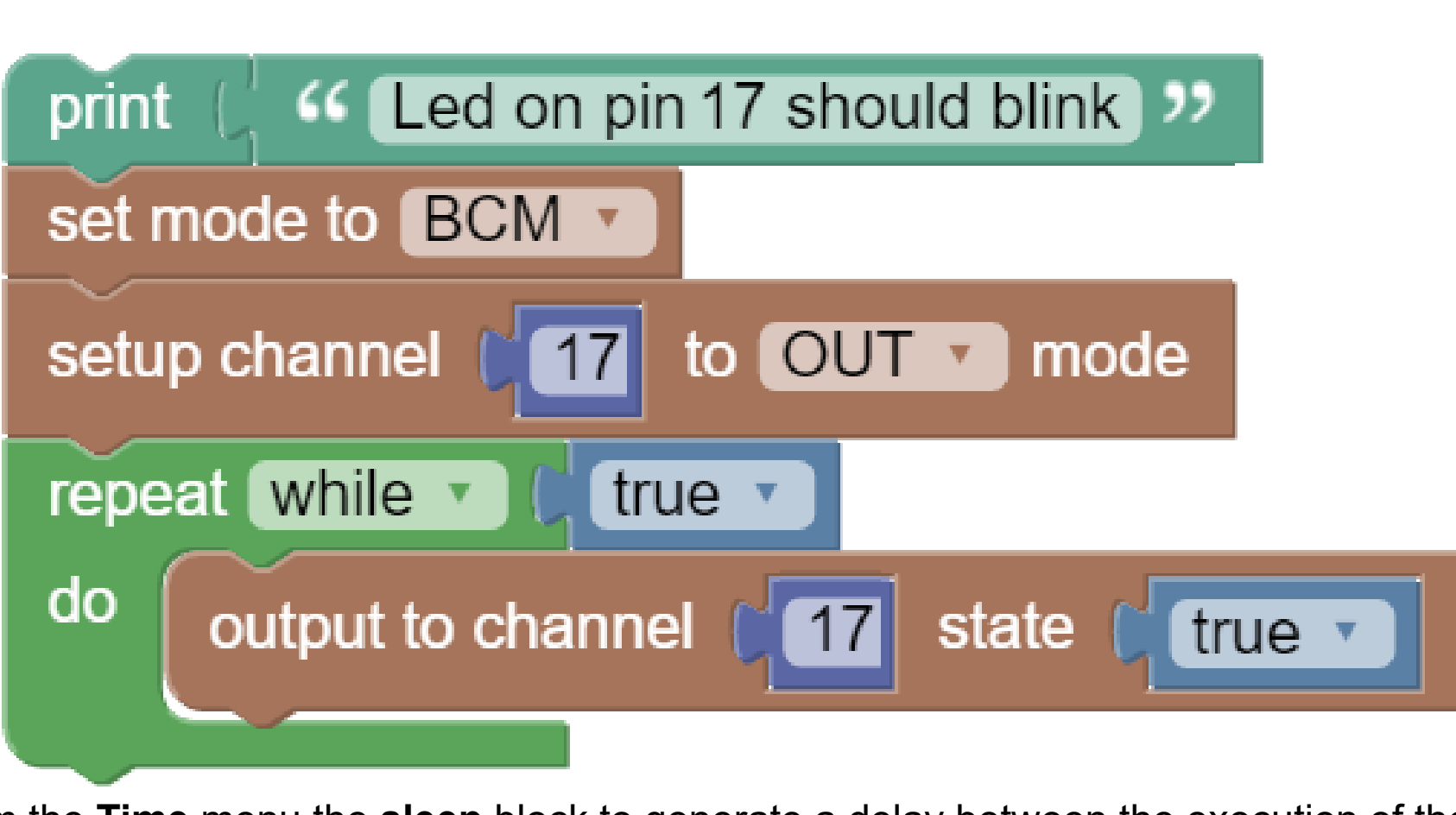
5. Add a **repeat while** loop by selecting **Program->Loops**, and move it to the canvas.



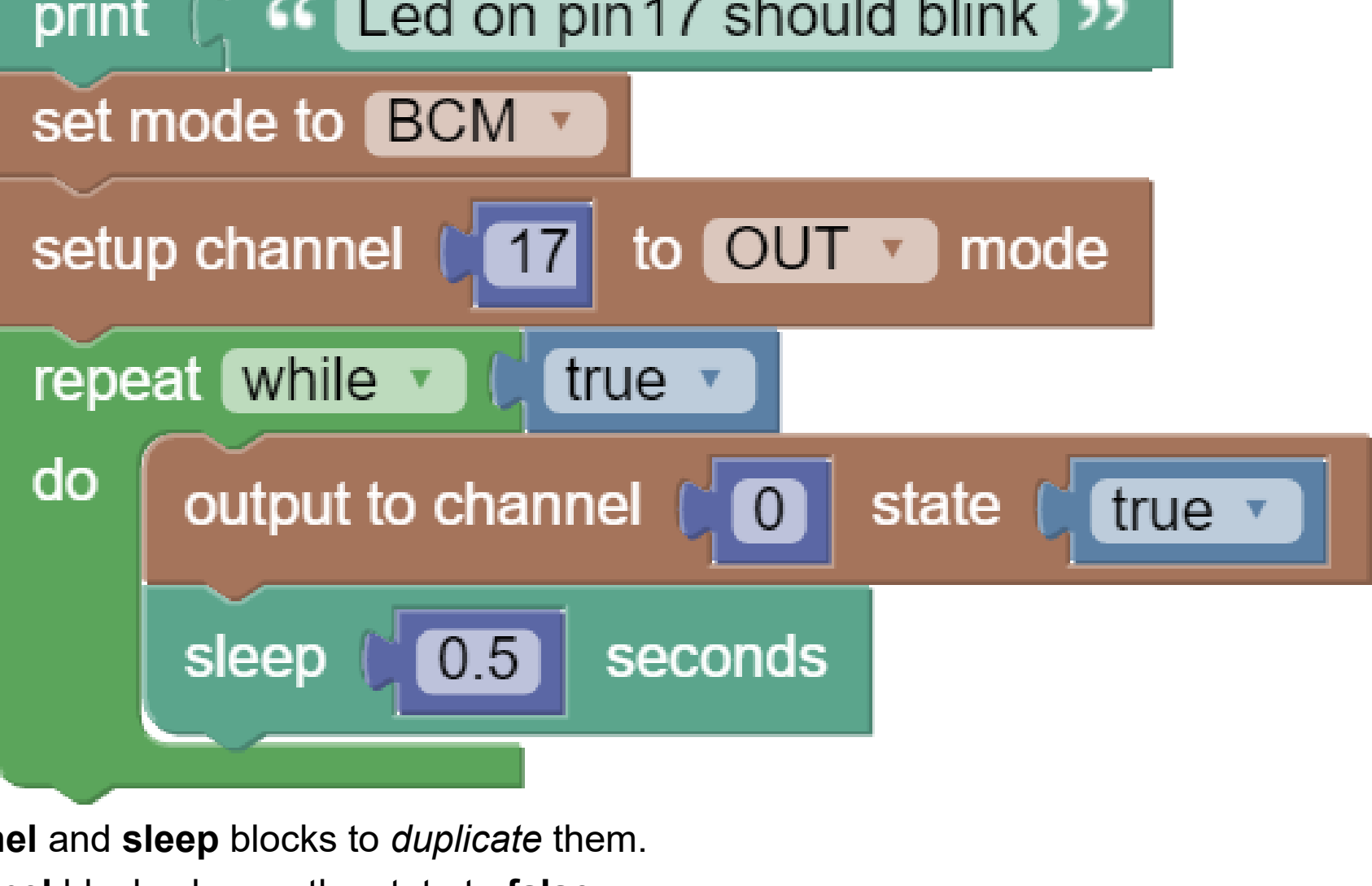
6. Select **Program->Logic** and drag the **true** block to the end of the repeat **while** block.



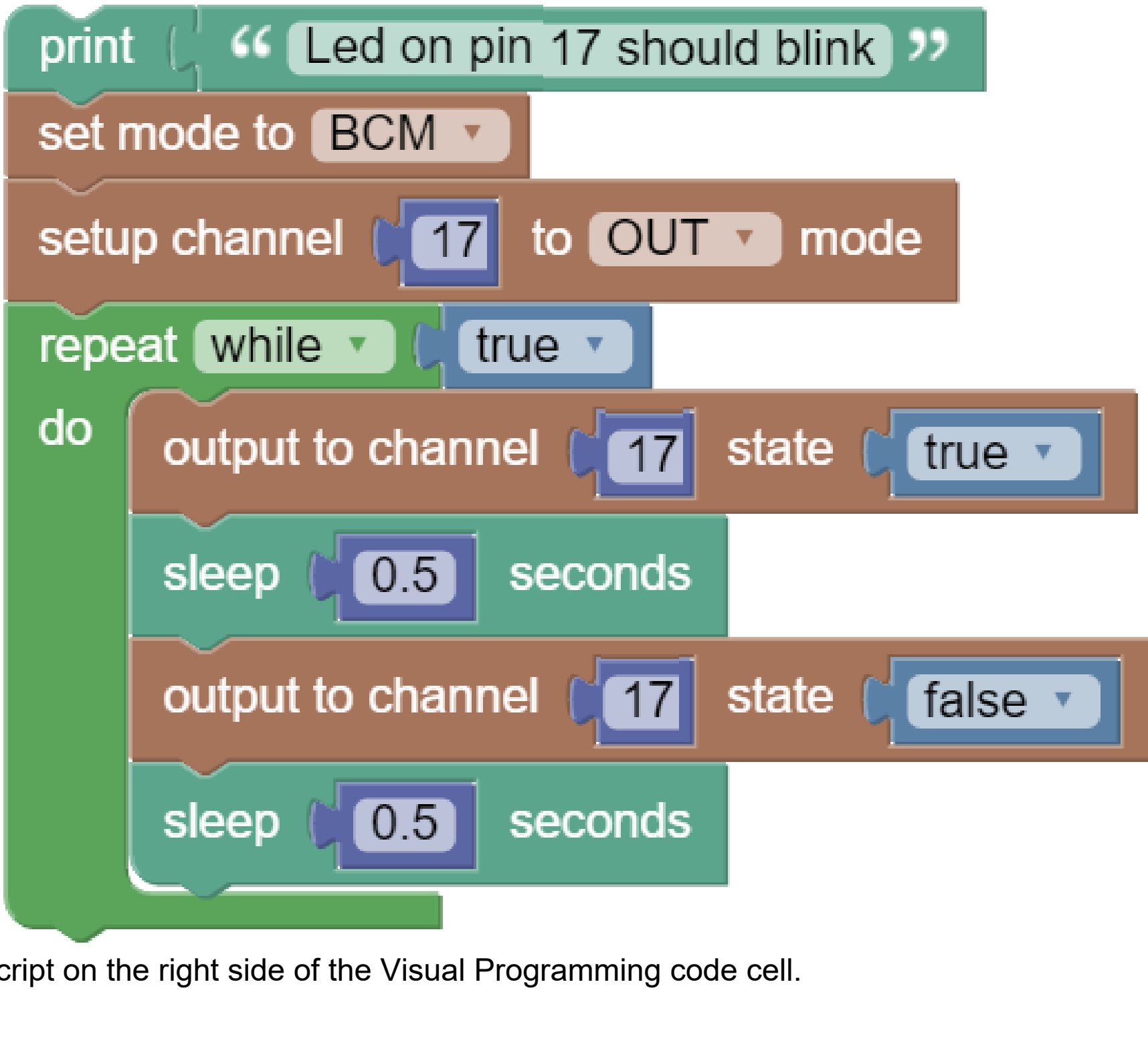
7. Add a **output to channel** block by selecting **Pin Access**. Drag the **output to channel** block inside the repeat while loop. From the **Program->Logic** menu, select the **true** block and add it to the **output to channel** block. This will turn on the GPIO pin and provide 3.3V to power the LED.



8. Add a delay block by selecting from the **time** menu the **sleep** block to generate a delay between the execution of the blocks. Drag it inside the repeat while loop below the **output to channel** block. Set the delay to 0.5 seconds.

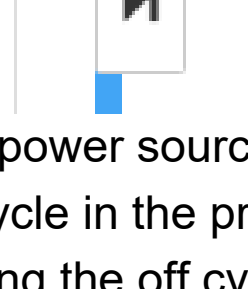


10. For the duplicated **output to channel** block, change the state to **false**.



- #### Step 4: Run the Application.

start blinking.



- has terminated. Try restarting the application and stopping again during the off cycle.

International Morse Code

1. The length of a dot is one unit.
2. A dash is three units.
3. The space between parts of the same letter is one unit.
4. The space between letters is three units.
5. The space between words is seven units.

4. The space between letters is three units.
5. The space between words is seven units.

A B C D E F G H I J K L M N O P Q R

U V W X Y Z

1 2 3 4 5 6 7 8

S ● ● ● 9 ■ ■ ■ ■ ●
 T ■ 0 ■ ■ ■ ■ ■